

MeshWhisper

Messaging as Ambient Infrastructure

GestureLoop Ltd · meshwhisper.net · April 2026

Every developer faces the same choice

Build it yourself

WebSocket servers, offline queuing, push notifications, media handling. Each piece is a project on its own.

Cost: months of engineering

Pay for a service

Sendbird from \$349/mo.
PubNub from \$98/mo per 1k MAU. Costs scale aggressively.

Cost: four figures/month at scale

Use Firebase

Free at small scale. Every message passes through Google in readable form. GDPR. Vendor lock-in. Pricing cliff.

Cost: privacy and control

The trilemma has a structural cause: traditional infrastructure must *read* a message to route it.

MeshWhisper breaks that assumption.

“A relay does not need to read a message to forward it.”

If messages are encrypted before they leave the device,
a relay is just a pipe. It forwards an opaque blob.

The relay cannot read the content. It doesn't need to.

Two layers, one protocol

Node Layer — the backbone

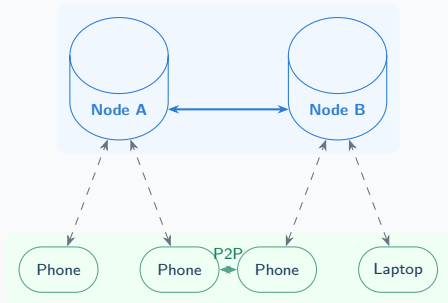
Deployed by each app developer. One Docker container, one port.

- Packet relay and store-and-forward
- Push notification forwarding (APNs / FCM)
- Encrypted media storage
- Prekey directory for first contact

Always on. Never holds a decryption key.
A post box, not a postman.

Device Layer — the mesh

Every SDK-embedded device relays for every other.



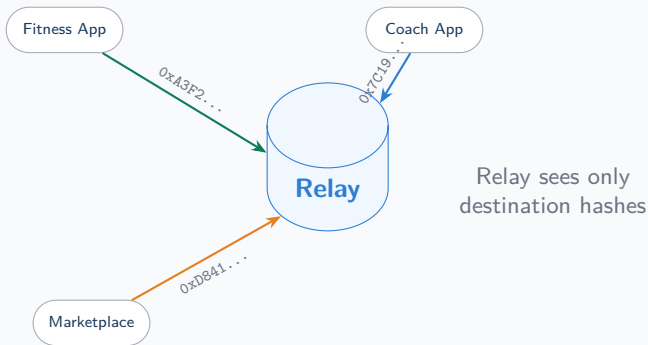
Integrate in an afternoon

Step	
Deploy	<code>docker run meshwhisper/node</code>
Install	<code>npm install @meshwhisper/sdk</code>
Init	<code>MeshWhisper.init({...})</code>
Send	<code>sendMessage(id, payload)</code>
Receive	<code>onMessage: msg => ...</code>

You own	MeshWhisper owns
User identity	Key exchange (PQXDH)
Message content	Session encryption
Your Node	Routing & relay
Your users' data	Store-and-forward
Your UI	Push dispatch

Post-quantum E2EE with zero cryptographic decisions required.

Namespace isolation — cryptographic, not configured



Destination hashes are computed as `BLAKE3(namespace_id || peer_key || epoch_hour)`.

A relay node routing packets for dozens of apps **cannot tell them apart**, cannot inject into any namespace, and cannot correlate traffic across applications — even under active observation.

Privacy that strengthens with scale

Centralised messaging

Scale **concentrates** metadata.

More users → more valuable surveillance target.

A single operator sees:

- Who messages whom
- When conversations are active
- Message frequency and timing

MeshWhisper

Scale **distributes** metadata.

More nodes → shorter each node's view.

Any single node sees only a fragment:

- Source IP and push token
- Opaque destination hash (rotates hourly)
- Traffic volume — obscured by chaff

Every device continuously generates **chaff** — constant-rate encrypted background traffic — so a node cannot distinguish real messages from noise or infer when conversations are active.

Current: Level 2 (PQXDH)

X3DH key exchange extended with ML-KEM-768 encapsulation.

Protects against *harvest now, decrypt later*: an adversary recording traffic today cannot decrypt it with a future quantum computer.

Same security class as Apple iMessage.

Roadmap: Level 3 (PQ3 ratchet)

Periodic ML-KEM injection into the Double Ratchet session itself.

A session *heals automatically* after compromise — exposure is bounded by the injection interval, not the lifetime of the session.

Beyond Signal. One of the only open-source SDKs at this security level.

Level 3 ratchet healing is especially valuable for long-lived machine sessions — IoT devices, AI agents — where sessions run indefinitely with no natural end.

Beyond human messaging

The protocol is content-blind. It moves encrypted blobs between key pairs.
That primitive applies wherever authenticated, asynchronous, E2EE delivery is needed.

Smart home / IoT

Devices become protocol peers. Commands are E2EE before leaving the phone.
Manufacturer's server is a relay, not a trusted intermediary with a record of your home's activity. Works on the local network when internet is down.

AI agent coordination

Agents authenticate each other via PQXDH — solving the impersonation problem most agentic frameworks leave open. Asynchronous delivery handles agents on different schedules.

Supply chain & logistics

Custody transfers, cold-chain records, delivery confirmations — between parties who don't share infrastructure and shouldn't trust each other's servers. Each party runs their own node.

Industrial & field systems

Sensors and controllers with intermittent connectivity get store-and-forward delivery, authenticated commands a relay cannot forge, and local routing when internet is unavailable.

Three stages of adoption

Stage 1 — Foundation (now)

GestureLoop deploys MeshWhisper in its own applications.

meshwhisper.net node live.
Prudence reference PWA.
Two live app integrations.

A developer evaluating the SDK finds a working relay network, not a proposal.

Stage 2 — Developer adoption

Other developers integrate the SDK and deploy nodes.

Multi-node privacy routing becomes real.

Messages route across independent operators.

Foundation's share of relay capacity shrinks as developer infrastructure grows.

Stage 3 — Mesh density

SDK-embedded user base grows across all applications.

Device-layer relay capacity becomes meaningful.

Multi-hop privacy becomes the typical case.

Each new user of any app improves delivery for every other app.

Each stage has independent value. The floor is set on day one.

Messaging's SMTP moment

SMTP, DNS, HTTP — protocols that became ambient infrastructure.

No single party operates them. No one controls them.

Applications use them without knowing their internals.

Messaging has not had this moment.

The dominant infrastructure is owned by a small number of platforms, each operating closed proprietary systems, each holding the plaintext of every message their users send.

MeshWhisper is an attempt to change that:

- Open protocol, MIT licensed

**Relay promiscuously,
connect selectively.**

```
npm install  
@meshwhisper/sdk
```

```
docker run  
meshwhisper/node
```

meshwhisper.net

Prudence demo app

`prudence.meshwhisper.net`

Get started

SDK	<code>npm install @meshwhisper/sdk</code>
Node	<code>docker run meshwhisper/node</code>
Docs	<code>meshwhisper.net</code>
Demo	<code>prudence.meshwhisper.net</code>
Contact	<code>anton@gestureloop.com</code>

MIT licensed · Self-hosting always free · GestureLoop Ltd, Ireland